



GitHub Trending Top 10

每日热门开源项目深度解读

2026-07-04

今日榜单

- | | | | |
|----|---|------------|-----|
| 1 | openai/codex-plugin-cc
Use Codex from Claude Code to review code or delegate tasks. | JavaScript | 2天 |
| 2 | JuliusBrussee/caveman
why use many token when few token do trick — Claude Code skill that cuts 65% o... | JavaScript | 3天 |
| 3 | alibaba/page-agent
JavaScript in-page GUI agent. Control web interfaces with natural language. | TypeScript | NEW |
| 4 | usestrix/strix
Open-source AI penetration testing tool to find and fix your app' s vulnerabiliti... | Python | 5天 |
| 5 | ChromeDevTools/chrome-devtools-mcp
Chrome DevTools for coding agents | TypeScript | 3天 |
| 6 | Zackriya-Solutions/meetily
Privacy first, AI meeting assistant with 4x faster Parakeet/Whisper live transcr... | Rust | NEW |
| 7 | asgeirtj/system_prompts_leaks
Extracted system prompts from Anthropic - Claude Fable 5, Opus 4.8, Claude Code,... | JavaScript | NEW |
| 8 | harvard-edge/cs249r_book
Machine Learning Systems | Python | 2天 |
| 9 | rommapp/romm
A beautiful, powerful, self-hosted rom manager and player. | Python | 2天 |
| 10 | ogulcancelik/herdr
agent multiplexer that lives in your terminal. | Rust | NEW |

#1

JavaScript

连续 2 天

codex-plugin-cc

Use Codex from Claude Code to review code or delegate tasks.

Stars **23,551** Forks **1,440** Today **+634**

<https://github.com/openai/codex-plugin-cc>

好的，这是对该开源项目的分析解读：

项目简介：这是一个由 OpenAI 官方开发的插件，允许用户在 Claude Code 的编程环境中直接调用 Codex 的能力。它解决了用户需要在不同AI工具间频繁切换的痛点，让你在 Claude 的界面里就能享受 Codex 的代码审查和任务委派功能。

核心功能： 1. **智能代码审查：**通过 `/codex:review` 命令，可以对当前未提交的代码或整个分支进行专业的代码审查。 2. **对抗性审查：**`/codex:adversarial-review` 命令能主动挑战你的设计决策和实现方案，帮助发现潜在风险与权衡问题。 3. **任务委派与后台管理：**支持将调试、修复等任务通过 `/codex:rescue` 委派给 Codex 在后台执行，并提供 `/codex:status` 和 `/codex:cancel` 来管理这些后台任务。

适用场景：主要适用于使用 Claude Code 进行开发的程序员。当需要第二双眼睛来审查代码、或者希望将某个复杂但独立的任务（如修复一个bug）交给另一个AI并行处理时，这个插件非常有用。

技术亮点：该项目巧妙地利用了 Claude Code 的插件系统，实现了两大AI模型间的无缝 workflow 集成。它通过定义一系列简洁的斜杠命令，将 Codex 强大的代码分析和执行能力封装为 Claude 环境下的子代理（subagent），实现了“模型即服务”的协作模式。

上手建议：如果你是 Claude Code 用户，只需在终端中依次运行 `/plugin marketplace add openai/codex-plugin-cc` 和 `/plugin install codex@openai-codex` 即可安装。安装后，先运行 `/codex:setup` 完成配置，就可以尝试使用 `/codex:review --background` 体验后台代码审查功能了。

#2

JavaScript

连续 3 天

caveman

why use many token when few token do trick — Claude Code skill that cuts 65% of tokens by talking like caveman

Stars **83,392** Forks **4,655** Today **+2,863**

<https://github.com/JuliusBrussee/caveman>

好的，这是对开源项目 **JuliusBrussee/caveman** 的深度解读。

项目简介

这个项目是一个能让AI编程助手“说人话”的插件，更准确地说，是让它“说原始人话”。它的核心目标是大幅削减AI回复中的“废话”和“客套话”，在保证技术信息和代码准确性的前提下，通过让AI模仿原始人的说话方式（类似“Why use many token when few do trick”），将输出Token数量减少高达65%，从而节省使用AI编程助手的成本和等待时间。

核心功能

- 一键安装与多Agent兼容：**提供一个命令行脚本，能自动检测你电脑上安装的30多种主流AI编程助手（如Claude Code、Cursor、Gemini等），并为他们统一安装此插件。
- 可调节的“原始人”等级：**内置了从lite（轻度精简）到ultra（极度精简）等多个说话风格等级，用户可以通过 `/caveman <等级>` 命令随时切换，灵活控制回复的简洁程度。
- 零技术损失的精简：**项目声称并经过基准测试验证，其在精简回复时，会保留所有代码、命令和错误信息等关键技术细节，只剔除冗余的修饰词和解释性文字，确保功能不受影响。
- 简易开关与持久化：**安装后，通过“talk like caveman”或“normal mode”等自然语言指令即可开启或关闭该模式。选择的“原始人”等级在当前会话中会持续生效，无需重复设置。

适用场景

任何频繁使用AI编程助手（如Claude Code、Cursor、Copilot等）的开发者都会是这个项目的目标用户。尤其是在进行大量代码生成、调试或咨询时，开启“原始人模式”可以显著减少Token消耗，从而节省API调用费用或等待时间，特别适合需要快速获取技术要点、不希望阅读长篇大论回复的场景。

技术亮点

项目的技术实现非常巧妙。它不是去修改AI模型本身，而是通过修改AI助手的**系统提示词（System Prompt）** 或**规则文件（Rule File）**，在AI生成回复之前就设定好其语言风格。这种做法成本低、效

果好、兼容性强，并且“原始人”风格的设定本身就是一个天才般的“Prompt Engineering”案例，它通过一种幽默且指令明确的方式，高效地约束了AI的输出格式。

上手建议

使用非常简单，直接打开终端，根据你的操作系统运行项目首页提供的安装命令（macOS/Linux用curl，Windows用PowerShell），等待约30秒即可完成。如果你是开发者想要贡献代码，可以阅读项目中的INSTALL.md文件了解不同Agent的安装细节，并查看其核心的规则文件（如CLAUDE.md）来理解其工作原理。

#3

TypeScript

NEW

page-agent

JavaScript in-page GUI agent. Control web interfaces with natural language.

Stars **22,677** Forks **1,966** Today **+1,110**

<https://github.com/alibaba/page-agent>

好的，这是对阿里巴巴开源项目 page-agent 的深度解读分析。

项目简介： Page Agent 是一个运行在网页中的“GUI 智能体”，它允许用户直接用自然语言来控制任何网页界面。简单来说，它就像给每个网页请了一个“AI 管家”，你说“帮我登录”，它就能自动找到按钮并点击。这个项目解决了传统网页自动化需要编写复杂脚本、依赖特定框架或浏览器插件的问题，将交互门槛降到了最低。

核心功能：

- 零侵入集成：** 无需安装浏览器插件、Python 环境或无头浏览器，只需在网页中引入一段 JavaScript 代码即可使用。
- 基于文本的 DOM 操作：** 它不依赖截图或多模态大模型，而是直接读取和操作网页的 DOM（文档对象模型）结构，这意味着运行速度更快，且无需额外的视觉识别权限。
- 自带模型，自由选择：** 项目支持接入各种大语言模型（如 GPT、Qwen 等），用户可以选择自己的 API Key，或者使用项目提供的免费测试 API 快速体验。
- 跨标签页扩展：** 通过可选的 Chrome 扩展，Page Agent 的能力可以跨越单个页面，实现跨标签页的复杂任务。

适用场景：

- SaaS 产品经理/开发者：** 可以快速为自己的产品添加一个“AI Copilot”，让用户用一句话完成复杂操作，无需重写后端逻辑。
- 企业员工/管理员：** 对于 ERP、CRM 等复杂的企业管理系统，可以用自然语言完成表单填写、数据查询等繁琐工作，极大提升效率。
- 无障碍技术开发者：** 可以为任何网页应用创建语音控制或屏幕阅读接口，让有障碍的用户也能轻松使用网络服务。

技术亮点： 该项目最亮眼的技术点是“文本化 DOM 操作”。它没有选择当下流行的“截图+视觉模型”方案，而是将网页元素的结构和属性解析成纯文本描述，再交给大模型去理解并决策。这种做法非常巧妙，既避免了对多模态大模型的依赖，降低了成本和延迟，又无需特殊权限，使得集成变得前所未有的轻量 and 简单。

上手建议： 想快速体验，最简单的方式是直接在 HTML 文件中插入一行 `<script>` 标签，即可在网页上唤起一个 AI 助手。对于开发者，推荐通过 NPM 安装，然后在代码中创建 PageAgent 实例，配置好

模型和 API Key 后，调用 `execute` 方法传入指令即可。如果想为项目做贡献，建议先阅读 `CONTRIBUTING.md` 和 `maintainer's note`，了解项目当前的开发原则和状态。

#4

Python

连续 5 天

strix

Open-source AI penetration testing tool to find and fix your app's vulnerabilities.

Stars **35,427** Forks **3,618** Today **+2,803**

<https://github.com/usestrix/strix>

好的，作为一名资深开源技术分析师，我来为你解读这个名为 `strix` 的开源项目。

项目简介： Strix 是一个开源的 AI 渗透测试工具，它利用 AI 代理模拟真实黑客的攻击行为，来自动化地发现和验证 Web 应用程序中的安全漏洞。其核心目标是解决传统安全测试中人工渗透测试成本高、周期长，以及静态代码分析工具误报率高的问题，为开发者和安全团队提供快速、准确的安全检测手段。

核心功能： 1. **自动化渗透测试：** Strix 内置了侦察、漏洞利用和验证的完整工具链，能够像真实黑客一样动态运行你的代码并发现漏洞。 2. **多代理协同工作：** 它支持组建多个 AI 渗透测试“特工”团队，可以并行工作并相互协作，从而大规模、高效地扫描整个应用。 3. **真实漏洞验证：** 与传统的漏洞扫描器不同，Strix 会生成可验证的漏洞利用代码（PoC），而不是仅仅给出一个可能存在风险的警告，有效减少了误报。 4. **自动修复与报告：** 它不仅能发现问题，还能自动生成安全补丁，并输出符合合规性要求的渗透测试报告，让修复工作更加便捷。

适用场景： - **应用安全团队：** 在进行常规或快速的应用安全测试时，可以用它来自动化替代部分人工渗透测试工作。 - **DevSecOps 流程：** 可以无缝集成到 CI/CD 流水线（如 GitHub Actions），在每次代码提交时自动扫描，在代码进入生产环境前拦截漏洞。 - **漏洞赏金猎人：** 可以自动化漏洞挖掘研究，快速生成 PoC 用于提交报告，提高效率。

技术亮点： - **Agentic 架构：** 项目最核心的亮点在于其基于“代理”（Agent）的设计。每个 AI 代理都像一个独立的黑客，拥有决策和行动能力，这比传统的基于规则或单一模型的扫描器要灵活和智能得多。 - **动态代码执行：** 它不是静态分析代码，而是通过 Docker 沙箱环境动态运行你的应用，从而发现运行时才能暴露的逻辑漏洞和配置问题，这是其能发现真实漏洞的关键。 - **多 LLM 支持：** 项目设计上不绑定特定的大语言模型，支持 OpenAI、Anthropic 等多种后端，这为用户提供了选择灵活性，并能利用不同模型的优势。

上手建议： 如果你想立即体验，最直接的方式是使用其提供的一键安装脚本 `curl -sSL https://strix.ai/install | bash`。你需要确保本地运行着 Docker，并准备好一个 LLM API 密钥。之

后，只需运行 `strix --target ./app-directory` 即可对你的应用目录进行首次扫描。想深入了解或贡献代码，可以访问其官方文档 docs.strix.ai 和 GitHub 仓库。

#5

TypeScript

连续 3 天

chrome-devtools-mcp

Chrome DevTools for coding agents

Stars **45,609** Forks **2,969** Today **+405**

<https://github.com/ChromeDevTools/chrome-devtools-mcp>

好的，这是对 ChromeDevTools/chrome-devtools-mcp 项目的深度解读。

项目简介： 这个项目是一个“桥梁”，它通过模型上下文协议（MCP），让AI编程助手（如 Claude、Cursor 等）能够直接操控和检查一个真实的 Chrome 浏览器。它解决了AI在自动化测试、调试和性能分析时，无法像人类开发者一样直接访问浏览器内部状态的问题，将 Chrome DevTools 的强大能力赋予了AI。

核心功能： 1. **AI驱动的性能分析：** AI可以像使用 Chrome 开发者工具的 Performance 面板一样，录制和分析页面性能，并给出优化建议。 2. **深度浏览器调试：** AI能检查网络请求、查看控制台日志（包括源码映射后的堆栈信息），以及截取页面截图，进行全面的问题诊断。 3. **可靠的浏览器自动化：** 基于 Puppeteer 实现，AI不仅能执行点击、输入等操作，还能智能等待操作结果，确保自动化流程的稳定性。 4. **灵活的运行模式：** 提供完整的“全功能模式”和轻量级的“--slim 模式”，后者适合只需基础浏览器操作的场景，资源占用更少。

适用场景： 主要面向使用AI编程助手进行开发的前端工程师和测试工程师。例如，在开发过程中，可以让AI助手自动打开页面，分析性能瓶颈；或在CI/CD流程中，让AI自动执行端到端测试并捕获错误日志。

技术亮点： 技术实现上，该项目巧妙地采用了 MCP 协议作为AI与浏览器之间的标准化通信接口，使得它可以无缝对接任何支持 MCP 的AI客户端。同时，它直接复用了 Chrome DevTools 的底层引擎，确保了功能的完整性和权威性，而非从零实现一个简化版。

上手建议： 使用非常简单，只需在你的 MCP 客户端（如 Claude for Desktop）的配置文件中，添加项目提供的 `JSON` 配置即可开始使用。对于想要贡献的开发者，建议先阅读项目文档中的 `CONTRIBUTING.md` 和 `design-principles.md`，了解其设计理念和开发规范。

#6

Rust

NEW

meetily

Privacy first, AI meeting assistant with 4x faster Parakeet/Whisper live transcription, speaker diarization, and Ollama summarization built on Rust. 100% local processing. no cloud required. Meetily (Meetly Ai -<https://meetily.ai>) is the #1 Self-hosted, Open-source Ai meeting note taker for macOS & Windows.

Stars **14,404** Forks **1,593** Today **+607**

<https://github.com/Zackriya-Solutions/meetily>

好的，作为一名资深的开源技术分析师，我来为你解读这个项目。

项目简介： Meetily 是一款以隐私为首要原则的 AI 会议助手。它完全运行在你的本地设备上，可以实时转录、区分说话人并总结会议内容，所有数据处理都不依赖云端，从根本上解决了企业会议数据泄露和合规风险问题。

核心功能： 1. **隐私优先的本地处理：** 所有 AI 模型（语音识别、说话人分离、总结）均在本地运行，数据不出设备，无需联网。 2. **超高速实时转录：** 基于 Rust 语言，集成了比传统 Whisper 快 4 倍的 Parakeet/Whisper 模型，实现低延迟的实时语音转文字。 3. **说话人分离：** 能够区分会议中的不同发言人，让转录文本清晰可辨。 4. **AI 智能总结：** 利用本地运行的 Ollama 模型，自动生成会议摘要、行动项等关键信息。

适用场景： 这个项目非常适合对数据隐私和合规性有严格要求的专业人士和团队，例如企业高管、法务、医疗工作者、咨询顾问以及需要记录和回顾大量会议的远程办公团队。它也是开源和自托管爱好者的理想选择。

技术亮点： 技术实现上，项目使用 Rust 语言开发，保证了高性能和内存安全。它巧妙地集成了多个本地 AI 模型：利用更快的 Parakeet/Whisper 模型进行实时转录，结合说话人分离技术，最后通过 Ollama 调用本地大模型进行总结，构建了一个完整且高效的本地化 AI 工作流。

上手建议： 想使用的话，可以直接从项目的 GitHub Releases 页面下载适用于 macOS 或 Windows 的预编译安装包。对于想贡献代码的开发者，可以 Fork 仓库，阅读其基于 Rust 的源码，并按照开发文档搭建本地开发环境。

#7

JavaScript

NEW

system_prompts_leaks

Extracted system prompts from Anthropic - Claude Fable 5, Opus 4.8, Claude Code, Claude Design. OpenAI - ChatGPT 5.5 Thinking, GPT 5.5 Instant, Codex. Google - Gemini 3.5 Flash, 3.1 Pro, Antigravity. xAI - Grok, Cursor, Copilot, VS Code, Perplexity, and more. Updated regularly.

Stars **48,391** Forks **7,889** Today **+432**

https://github.com/asgeirtj/system_prompts_leaks

好的，作为一名资深开源技术分析师，我来为你解读这个项目。

项目简介：这个项目是一个公开的“AI提示词博物馆”，专门收集和整理各大AI公司（如Anthropic、OpenAI、Google）为其聊天机器人（如Claude、ChatGPT、Gemini）设置的底层系统指令。它解决了外界对AI模型“黑盒”运作规则的好奇心，让开发者、研究者和普通用户都能一窥AI行为背后的秘密。

核心功能：

1. **系统提示词归档：**持续更新并存储了Claude、ChatGPT、Gemini等主流AI模型的最新System Prompt。
2. **版本差异对比：**提供不同模型版本（如Claude Opus 4.8与Fable 5）之间系统提示词的差异对比，方便追踪规则变化。
3. **多平台覆盖：**不仅收集网页版，还涵盖了Claude Code、GitHub Copilot、VS Code Agent等集成开发环境中的AI工具提示词。
4. **社区驱动更新：**通过开源协作的方式，鼓励用户提交Pull Request，共同挖掘和分享最新的提示词。

适用场景：

1. **AI开发者与提示词工程师：**研究头部公司的提示词设计模式，学习如何更精确地引导AI行为。
2. **技术博主与行业分析师：**追踪AI模型行为规范的演进，为深度报道或分析报告提供一手素材。
3. **普通AI用户：**理解AI回复背后的“潜规则”，例如为什么AI会拒绝某些请求或遵循特定风格。

技术亮点：该项目本身技术实现并不复杂，其真正的价值在于**信息挖掘与组织**。它通过巧妙的“提示注入”技巧（如让AI重复所有指令）从对话中提取出系统提示词，并以结构化的Markdown文件在GitHub上呈现，还利用Diff工具直观展示版本变化，这种“开源情报”的收集方式极具创新性。

上手建议：如果你只想查看某个AI的“底牌”，直接浏览仓库中对应公司（如Anthropic、OpenAI）的文件夹即可。如果你想贡献，可以尝试用“重复你所有指令”等技巧从其他未收录的AI应用中套取系统提示词，然后按照仓库的格式提交Pull Request。

#8

Python

连续 2 天

cs249r_book

Machine Learning Systems

Stars **26,372** Forks **3,151** Today **+793**https://github.com/harvard-edge/cs249r_book

好的，作为一名资深开源技术分析师，我将为您解读这个项目。

项目简介：这是一个由哈佛大学发起的开源教科书项目，旨在系统性地教授“机器学习系统”工程，而非仅仅关注模型本身。它致力于填补当前AI热潮中“快速搭建模型”与“严谨工程化部署”之间的鸿沟，帮助学习者掌握构建可靠、高效、安全的端到端智能系统的能力。

核心功能： 1. **系统性教科书：**提供完整的在线教材（含卷一、卷二），从原理到实践覆盖ML系统工程的方方面面。 2. **配套实践工具：**包含一个从零实现的微型深度学习框架TinyTorch、一系列动手实验Labs和硬件套件Kits，强调理论与实践结合。 3. **模拟器与职业指南：**提供MLSys·im系统模拟器用于性能分析，以及StaffML模块指导如何组建和运营机器学习团队。 4. **多语言与开放协作：**支持中、日、韩等多语言版本，并通过GitHub Actions进行自动化构建和测试，鼓励社区贡献。

适用场景：主要面向高校学生（特别是研究生和高年级本科生）、AI领域的软件工程师、算法工程师以及技术管理者。适用于希望系统学习如何将机器学习模型从研究原型部署到生产环境，并优化其性能、可靠性和安全性的任何开发者。

技术亮点：该项目最值得关注的是其“教学即代码”的理念。它通过TinyTorch这种极简但功能完整的框架，让学习者能像阅读代码一样理解深度学习框架的内部机制。同时，项目集成了大量的CI/CD自动化流程，确保教材、代码和实验环境的持续正确性，这本身就是ML系统工程的最佳实践。

上手建议：强烈建议从阅读在线教科书mlsysbook.ai开始，了解整体知识框架。之后可以顺序进行TinyTorch的代码阅读和Labs中的动手实验，从实现一个简单的神经网络到在真实硬件（如Arduino套件）上运行，完成从理论到实践的闭环。

#9

Python

连续 2 天

romm

A beautiful, powerful, self-hosted rom manager and player.

Stars **9,951** Forks **482** Today **+239**

<https://github.com/rommapp/romm>

好的，这是对 RomM 项目的深度解读。

项目简介： RomM 是一个自托管的复古游戏 ROM 管理器与播放器，旨在解决游戏爱好者管理分散的 ROM 文件、获取元数据以及在多设备上统一游玩体验的痛点。它提供了一个美观的 Web 界面，让你可以像管理数字音乐或电影一样，轻松地浏览、搜索和启动你的整个复古游戏库。

核心功能：

- 1. 自动刮削与美化：**能自动从 IGDB、Screenscraper 等多个数据库抓取游戏封面、描述、评分等元数据，并支持自定义艺术图，让杂乱的文件变成精美的数字图鉴。
- 2. 浏览器内直接游玩：**集成了 EmulatorJS 和 RuffleRS 等模拟器，用户无需下载和配置任何本地模拟器，即可在现代浏览器中直接启动并游玩大部分复古游戏。
- 3. 强大的文件管理：**原生支持多碟游戏、DLC、Mod、汉化补丁等复杂文件结构，并能根据文件名中的标签（如 [USA], [HACK]）进行智能解析和筛选。
- 4. 多平台与权限共享：**支持超过400个游戏平台，并允许你创建受限的访客账户，安全地与朋友分享你的游戏库，或仅授权特定用户管理。

适用场景：

- **复古游戏收藏家：**拥有大量 ROM 文件，希望有一个统一、美观、可搜索的“数字博物馆”来管理和展示自己的收藏。
- **家庭或朋友间的游戏分享者：**想在家里搭建一个游戏服务器，让家人或朋友通过手机、平板、电视等任何有浏览器的设备随时游玩。
- **掌机/盒子玩家：**配合社区开发的客户端（如 Grout、Argosy），可以将 RomM 作为游戏库的后端，方便在 muOS、Android 等系统上远程下载和同步游戏。

技术亮点：

- **全栈自托管，架构清晰：**项目采用 Python 后端（可能基于 FastAPI 或 Flask）配合现代前端框架，提供了从扫描、元数据管理到游戏流式传输的完整解决方案，易于通过 Docker 部署。
- **高度可扩展的插件生态：**不仅仅是一个独立应用，它还通过官方 API 和社区贡献，衍生出了覆盖 Playnite、Android、iOS、Switch 等不同平台的客户端，形成了一个丰富的生态。
- **智能的元数据聚合与文件解析：**并非简单调用一个 API，而是整合了多个元数据源，并设计了复杂的文件名解析逻辑，能处理不同命名规范和标签，体现了对复古游戏社区习惯的深刻理解。

上手建议：

- **快速体验：**最直接的方式是参考官方文档的“快速入门指南”，使用 Docker Compose 一键部署。你只需要准备一个存放 ROM 文件的目录，并配置好一个数据库（如 SQLite 或 PostgreSQL）。
- **贡献代码：**项目由 Python 驱动，如果你想贡献代码，可以先从 GitHub 的 Issues 看起，寻找标记为“good first issue”的任务。同时，仔细阅读项目根目录的 CONTRIBUTING.md 贡献指南是关键。
- **加入社区：**如果遇到问题或有想法，最推荐的方式是加入项目的 Discord 服务器。那里面有活跃的开发者和其他用户，能快速获得帮助和反馈。

#10

Rust

NEW

herdr

agent multiplexer that lives in your terminal.

Stars **11,065** Forks **649** Today **+478**

<https://github.com/ogulcancelik/herdr>

好的，这是对开源项目 herdr 的深度解读。

项目简介： herdr 是一个运行在你终端里的“AI代理多路复用器”。它解决了同时管理多个AI编码代理时，难以直观监控每个代理状态（如阻塞、工作中、完成）和无法在断开连接后保持代理持续运行的核心痛点。

核心功能：

- 真实终端视图：** 每个AI代理都拥有一个独立的、真实的终端，而非模拟终端，因此能完美支持各种命令行界面（TUI）程序。
- 代理状态总览：** 侧边栏会清晰显示每个代理的状态（阻塞、工作、完成、空闲），让你一眼掌握所有代理的“健康状况”。
- 持久化会话：** 即使关闭笔记本电脑，后台服务器也会保持所有代理和窗口运行，你可以随时从任何终端（包括手机SSH）重新连接。
- 灵活的组织方式：** 支持工作区、标签页和窗格，你可以像操作现代终端模拟器一样，通过鼠标拖拽来组织代理界面。

适用场景： 任何需要同时运行和监控多个AI编码代理的开发者或团队。例如，在本地或远程服务器上，同时运行一个负责代码审查、一个负责编写测试、一个负责重构代码的代理，并通过herdr在一个界面里统一管理它们。

技术亮点： 项目使用 Rust 语言编写，编译为单个约10MB的轻量级二进制文件，无任何外部依赖，可在 Linux、macOS（Windows测试版）上原生运行。其核心架构是一个后台守护进程，确保了会话的持久性，并通过本地Socket API和CLI提供了强大的可脚本化能力，让代理本身也能驱动herdr。

上手建议： 使用非常简便，在终端执行 `curl -fsSL https://herdr.dev/install.sh | sh` 即可安装，然后直接运行 `herdr` 命令。如果你想深入了解或贡献代码，可以从其官方文档网站（herdr.dev）和GitHub仓库的README开始。

数据来源: github.com/trending

报告日期: 2026-07-04

由 DeepSeek AI 提供智能分析 | 自动生成